

Kol HaNitzachon Phone System

Complete Functional Testing, Frontend CRUD, Audio Recording, and Azure Upload Guide

For developer verification, client acceptance testing, and GitHub documentation

System purpose	Allow callers to select recipients, enter donations, and pay through Cardknox (Sola).
Audio delivery	Menus and dynamic prompts use Azure-hosted recordings and Twilio/SignalWire <Play>.
Admin frontend	Single HTML page for recipient add, edit, delete, and name-recording upload/update.
Card storage rule	Raw card information and saved payment methods must not be retained after processing.

1. Scope and Important Testing Notes

This guide combines the current recipient-chain and number-audio tests with the frontend recipient-management tests and the complete client menu requirements. It should be used as an acceptance checklist, not as proof that every menu branch is already implemented.

- Use test data only. Never use a real customer card during development.
- Use the same CallSid throughout one IVR test session.
- Use a new CallSid when starting a separate scenario.
- Keep the API output/log window open during testing.
- Do not commit secrets.json, Azure SAS credentials, SignalWire tokens, or the Sola API key to GitHub.
- The actual recipient name MP3 and all required prompt MP3 files must exist in Azure for audible end-to-end testing.

2. Required Configuration

2.1 Sola / Cardknox test configuration

Store the API key in .NET User Secrets. Keep only non-secret values in appsettings.json.

```
"Sola": {  
  "ApiKey": "",  
  "SoftwareName": "WixSystem",  
  "SoftwareVersion": "1.0",  
  "RecurringApiMinorVersion": "2.1"  
}
```

User Secrets should contain:

```
"Sola": {  
  "ApiKey": "<TEST_API_KEY_PROVIDED_BY_THE_TEAM>"  
}
```

2.2 Azure Blob structure

```
ivrrecordings/  
  audio/  
    MainMenu/  
    Option1/  
    Option1-1/  
    Option2/  
    Recipients/  
    numbers1-1000/  
    charge-successful.mp3  
    charge-not-successful.mp3  
    minimum-donation.mp3
```

When the database stores a relative recipient path such as Recipients/25-test-recipient.mp3, the application should generate the corresponding Azure path audio/Recipients/25-test-recipient.mp3.

3. Test Data Preparation

3.1 Active recipient - happy path

```
{  
  "code": 25,  
  "name": "Test Recipient",  
  "nameRecordingUrl": "Recipients/25-test-recipient.mp3",  
  "startDate": "2026-07-01T00:00:00",  
  "endDate": "2026-12-31T23:59:59"  
}
```

Upload the matching file to:

```
audio/Recipients/25-test-recipient.mp3
```

3.2 Error-test recipients

Inactive recipient: code 30, EndDate in the past
Future recipient: code 31, StartDate in the future
Missing-audio recipient: code 32, active dates, NameRecordingUrl points to a non-existent blob

4. Frontend HTML Admin Page Testing

Open the single-page admin frontend while the API is running. Record the browser URL and API base URL used for the test.

4.1 Page-load and list test

1. Open the HTML admin page in a browser.
2. Confirm the recipient table/list loads without JavaScript errors.
3. Confirm each record displays ID/code, name, recording information, start date, end date, and action buttons.
4. Open browser Developer Tools > Network and confirm GET /api/recipients returns HTTP 200.
5. Refresh the page and confirm the list remains consistent.

4.2 Add recipient and upload recording

6. Click Add Recipient or open the empty recipient form.

7. Enter a unique code, recipient name, active StartDate, and active EndDate.
8. Attach a small valid MP3 for the name recording.
9. Click Save/Add.
10. Confirm the UI shows a success message and the new row appears.
11. Confirm the backend POST request returns 200/201.
12. Confirm the returned/saved NameRecordingUrl is not empty.
13. Open the generated recording URL or Azure blob and verify the MP3 is accessible and correct.
14. Call GET /api/recipients and verify the database values match the form.

Check	Expected result	Pass/Fail
Required fields	Blank name/code/dates are rejected with a clear message.	
File type	Non-audio or unsupported file is rejected.	
Duplicate code	A duplicate recipient code is rejected or handled according to the API rule.	
Azure upload	Uploaded MP3 exists and NameRecordingUrl points to it.	
Persistence	Recipient still exists after refresh.	

4.3 Edit recipient details

15. Choose an existing test recipient and click Edit.
16. Change the name and one date while keeping the recipient active.
17. Save the changes.
18. Confirm the PUT /api/recipients/{id} request returns 200/204.
19. Refresh the page and verify the changes persisted.
20. Call GET /api/recipients/{id} and verify the updated fields.

4.4 Update/replace the name recording

21. Edit the active test recipient.
 22. Attach a different valid MP3 file.
 23. Save the recipient.
 24. Confirm the upload request succeeds and NameRecordingUrl is updated or intentionally reuses the same path.
 25. Open the resulting Azure recording URL and confirm the new audio is served, not the old file.
 26. Run the IVR recipient happy path and verify the updated name recording is referenced in the XML response.
- Also test cancelling the edit without selecting a new recording; the old NameRecordingUrl should remain unchanged.
 - If the implementation creates a new blob path, verify whether the old blob should be retained or removed according to the project policy.

4.5 Delete recipient

27. Create a disposable test recipient.
28. Click Delete.
29. Confirm the confirmation dialog appears, if implemented.
30. Confirm deletion.
31. Verify DELETE /api/recipients/{id} returns 200/204.
32. Refresh the page and confirm the record is gone.
33. Call GET /api/recipients/{id}; expect 404.
34. Test its code through recipient-selection; expect recipient-not-found playback.

4.6 Frontend negative tests

- Save with blank name.
- Save with blank or invalid code.

- Save with EndDate earlier than StartDate.
- Upload an unsupported file type.
- Upload an oversized file, if a limit exists.
- Disconnect or stop the API and verify the page displays a useful error instead of silently failing.
- Double-click Save and verify duplicate submissions are prevented or safely handled.

5. Required Azure Recordings

5.1 Option 2 recipient-chain prompts

```
audio/Option2/enter-recipient-code.mp3
audio/Option2/recipient-not-found.mp3
audio/Option2/recipient.mp3
audio/Option2/has-been-competing-for.mp3
audio/Option2/days.mp3
audio/Option2/code-number-to-sponsor.mp3
audio/Option2/enter-pledge-amount.mp3
```

5.2 Base number recordings

```
audio/numbers1-1000/zero.mp3
one.mp3 ... nineteen.mp3
twenty.mp3, thirty.mp3, forty.mp3, fifty.mp3, sixty.mp3, seventy.mp3, eighty.mp3, ninety.mp3
hundred.mp3
thousand.mp3
and.mp3
```

The application composes numbers. Example: 25 = twenty.mp3 + five.mp3; 105 = one.mp3 + hundred.mp3 + and.mp3 + five.mp3.

6. IVR Happy-Path Test - Specific Recipient by Code

Use the same values in every request:

```
CallSid: TEST-RECIPIENT-001
From: +15551234567
```

Step	Endpoint	Input	Expected
Open main menu	POST /api/ivr/handle-call? step=main	Digits blank	Main-menu <Gather> and <Play> elements.
Choose specific recipient	POST /api/ivr/handle-call? step=main	Digits 2	Prompt for recipient selection/code flow.
Enter active code	POST /api/ivr/handle-call? step=recipient-selection	Digits 25	Full recipient recording chain; next action donation-amount.
Inspect session	GET /api/ivr/debug/session/TEST-RECIPIENT-001	N/A	RecipientId and RecipientCode populated; CurrentStep donation-amount.
Enter amount	POST /api/ivr/handle-call? step=donation-amount	Digits 5	Donation confirmation.
Confirm	POST /api/ivr/handle-call? step=confirm-donation	Digits 1	Card-number prompt.
Card number	POST /api/ivr/handle-call? step=payment-card-number	Approved test card digits	Expiry prompt.
Expiry	POST /api/ivr/handle-call? step=payment-expiry	MMYY	CVV prompt.

CVV	POST /api/ivr/handle-call?step=payment-cvv	Approved test CVV	Billing ZIP prompt.
Billing ZIP	POST /api/ivr/handle-call?step=payment-zip	Approved 5-digit test ZIP	Automatic Sola processing and success/failure response.

6.1 Expected recipient-chain playback order

Option2/recipient.mp3
 Recipients/25-test-recipient.mp3
 Option2/has-been-competing-for.mp3
 <number clips for days>
 Option2/days.mp3
 Option2/code-number-to-sponsor.mp3
 twenty.mp3
 five.mp3
 Option2/enter-pledge-amount.mp3

7. IVR Error and Resilience Tests

Scenario	How to test	Expected
Unknown code	recipient-selection, Digits 99999	recipient-not-found.mp3 and redirect to recipient-selection.
Inactive recipient	Use code 30	Same not-found behavior.
Future recipient	Use code 31	Same not-found behavior.
Missing recipient MP3	Use code 32	XML may be produced; opening the recording URL should show missing blob. Log/document asset failure.
Missing number MP3	Temporarily remove twenty.mp3	XML references it, but playback fails; confirms asset issue rather than lookup issue.
Invalid menu digit	main, Digits 9	Invalid option and return to main menu.
Missing digits	recipient-selection, blank Digits	Repeat recipient-code prompt.
Invalid card length	payment-card-number, invalid digits	Invalid-card prompt, no Sola charge.
Invalid expiry	expired or malformed MMY	Invalid-expiry prompt.
Invalid CVV	wrong length/non-numeric	Invalid-CVV prompt.
Invalid ZIP	not exactly five digits	Invalid-ZIP prompt.
Sola decline	Approved decline test card	Payment failure response, no success state.
API/network error	Temporarily use invalid gateway config in a safe local environment	Friendly failure response; no sensitive details spoken or logged.

8. Full Client Menu Requirements - Acceptance Matrix

This section translates the client instructions into testable acceptance criteria. Mark each row as Implemented, Partially Implemented, Pending, or Not Applicable after comparing it with the current branch.

Requirement	Description	Acceptance test	Status
Main menu	Announce active contestant count; options 1, 2, and 3.	Dynamic active count and all three branches work.	
Option 1.1	Per contestant, per competition day pledge.	Calculate total using each active contestant day count and announce total.	
Option 1.2	Fixed amount for every contestant.	Calculate total = amount x active contestant count and announce total.	
Option 2 submenu	Choose by code or from spoken list.	Both sub-options are available.	
Option 2.1	Enter contestant code.	Recipient is found; name, day count, and code are	

		played; donation proceeds.	
Option 2.2	Hear/select contestant from list.	Spoken list maps names to codes and accepts selection.	
Option 3	Hear full list of active contestants.	Every active contestant name, days, and sponsor code are chained.	
Return from option 3	Enter code and return to sponsorship flow.	Code entry continues into specific-recipient donation.	
Payment	Charge in Cardknox/Sola.	Successful charge produces transaction/intent ID.	
No card saving	Card should not be saved.	SetAsDefault false; detach or no persistent saved method; raw card/CVV cleared.	
Recordings	Menus use Azure audio and Play.	Generated TwiML/XML uses <Play>, not TTS for finalized prompts.	
Frontend	Add/edit/delete recipients and upload recording.	All CRUD and recording replacement tests pass.	

9. Security and Data-Handling Verification

- Card number, CVV, expiry, and billing ZIP are not written to logs.
- Raw card number and CVV are cleared immediately after tokenization or on failure.
- The card is not set as the default saved payment method.
- No API keys, SAS tokens, or SignalWire credentials are committed.
- Debug session endpoints are unavailable outside Development.
- Azure SAS URLs have the minimum required permissions and reasonable expiry.
- Only test credentials and approved gateway test cards are used.

10. GitHub Pre-Push Checklist

```
git status
git diff --stat
git grep -n "PSK_"
git grep -n "testdev"
git grep -n "SharedAccessSignature"
```

- Build: 0 errors.
- Frontend CRUD tested.
- Recording upload and replacement tested.
- Invalid and valid recipient selection tested.
- Generated Azure audio URLs opened successfully.
- Payment prompt sequence tested.
- No secrets staged.
- Documentation included under /docs.

```
git add .
git commit -m "Implement recipient chain, number audio composition, and admin recording workflow"
git push origin <branch-name>
```

11. Suggested Client Update

Hi Yanky,

I completed and pushed the missing number-recording and recipient chain-recording implementation. The IVR can

now select an active recipient by code, chain the recipient name and number recordings for the day count and sponsor code, and continue into the donation and payment flow.

I also included a testing guide covering the frontend recipient add/edit/delete workflow, recipient recording upload and replacement, Azure audio paths, the IVR happy path, and error scenarios. The remaining acceptance step is to run the guide with the actual recipient records and final recordings uploaded to Azure.

Best regards,
Ronald